

## Program-2: Develop a Multi-Stage Dockerfile for Container Orchestration

---

### Step 1: Create Project Directory

```
mkdir Program2  
cd Program2
```

### Step 2: Create Dockerfile (Multi-Stage)

```
nano Dockerfile
```

Add the following content:

```
### Start the Multi-Stage Dockerfile (Build & Production)
```

```
# ----- Stage 1: Build Stage -----  
FROM node:20-alpine AS builder
```

```
# Set working directory  
WORKDIR /app
```

```
# Copy package files and install dependencies  
COPY package.json package-lock.json ./  
RUN npm install
```

```
# Copy application source code  
COPY . .
```

```
# Build the application  
RUN npm run build
```

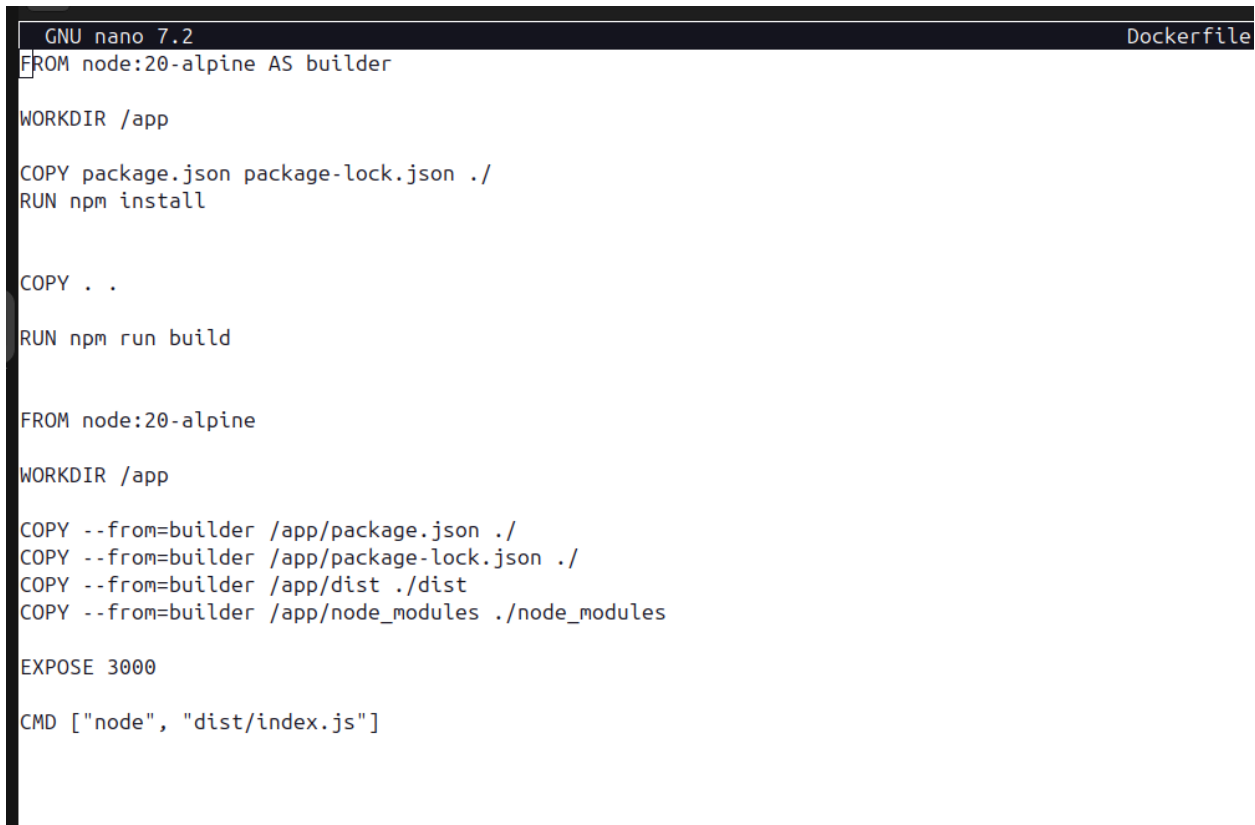
```
# ----- Stage 2: Production Stage -----  
FROM node:20-alpine
```

```
# Set working directory  
WORKDIR /app
```

```
# Copy only required files from build stage
COPY --from=builder /app/package.json ./
COPY --from=builder /app/package-lock.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
```

```
# Expose application port
EXPOSE 3000
```

```
# Start the application
CMD ["node", "dist/index.js"]
```

A screenshot of a terminal window showing the nano text editor editing a file named 'Dockerfile'. The editor's title bar shows 'GNU nano 7.2' on the left and 'Dockerfile' on the right. The content of the file is a Dockerfile with the following instructions: 'FROM node:20-alpine AS builder', 'WORKDIR /app', 'COPY package.json package-lock.json ./', 'RUN npm install', 'COPY . .', 'RUN npm run build', 'FROM node:20-alpine', 'WORKDIR /app', 'COPY --from=builder /app/package.json ./', 'COPY --from=builder /app/package-lock.json ./', 'COPY --from=builder /app/dist ./dist', 'COPY --from=builder /app/node\_modules ./node\_modules', 'EXPOSE 3000', and 'CMD ["node", "dist/index.js"]'. The cursor is positioned at the end of the first line.

```
GNU nano 7.2 Dockerfile
FROM node:20-alpine AS builder

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm install

COPY . .

RUN npm run build

FROM node:20-alpine

WORKDIR /app

COPY --from=builder /app/package.json ./
COPY --from=builder /app/package-lock.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules

EXPOSE 3000

CMD ["node", "dist/index.js"]
```

Save and exit.

## Step 3: Create Source Folder and index.js

```
mkdir src
nano src/index.js
```

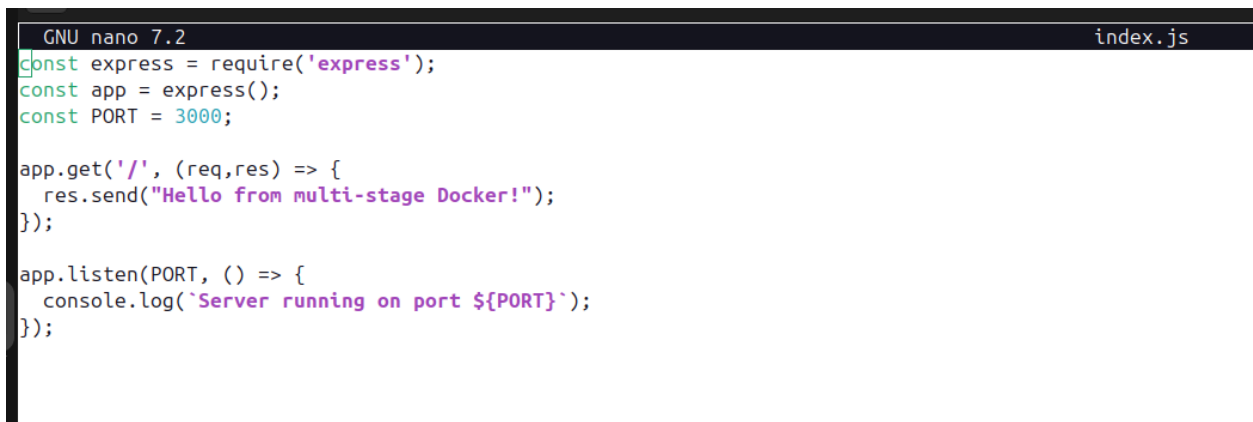
Add:

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello from multi-stage Docker!');
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Save and exit.



```
GNU nano 7.2 index.js
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send("Hello from multi-stage Docker!");
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

---

## Step 4: Create package.json

Run:

```
npm init -y
```

## Step 5: Install Node Modules

```
npm install
```

## Step 6: Build Docker Image

## docker build -t program2 .

```
1RV24MC109_syeda_ninra@syedaninra-IdeaPad-Gaming-3-15IMH05:~/program2$ docker build -t program2 .
ERROR: permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Head "http://%2Fvar%2Frun%2Fdocker.sock/_ping": dial unix /var/run/docker.sock: connect: permission denied
1RV24MC109_syeda_ninra@syedaninra-IdeaPad-Gaming-3-15IMH05:~/program2$ sudo docker build -t program2 .
[+] Building 171.9s (15/15) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 776B 0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine 3.6s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 2.31MB 0.1s
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:6178e78b9 25.4s
=> => resolve docker.io/library/node:20-alpine@sha256:6178e78b972f79c335 0.0s
=> => sha256:da04d522c98fe12816b2bcddf8413fca73645f8fa60f287 444B / 444B 0.7s
=> => sha256:6178e78b972f79c335df281f4b7674a2d85071aae2a 7.67kB / 7.67kB 0.0s
=> => sha256:be8d32d651b3e0c9c2b28fddc1d3888408125d703232 1.72kB / 1.72kB 0.0s
=> => sha256:2b56f2779663b9e1a77bdb5235dc31f1a81e534ccab 6.42kB / 6.42kB 0.0s
=> => sha256:60e45a9660cf8ebbbac9bba98180aa28b3966b7f 42.75MB / 42.75MB 24.0s
=> => sha256:e74e4ed823e9560b3fe51c0cab47dbdfdc4b1245360 1.26MB / 1.26MB 5.4s
=> => extracting sha256:60e45a9660cf8ebbbac9bba98180aa28b3966b7f2462d132 1.1s
=> => extracting sha256:e74e4ed823e9560b3fe51c0cab47dbdfdc4b124536043194 0.0s
=> => extracting sha256:da04d522c98fe12816b2bcddf8413fca73645f8fa60f287c 0.0s
=> [builder 2/6] WORKDIR /app 0.2s
=> [builder 3/6] COPY package-lock.json ./ 0.1s
=> [builder 4/6] RUN npm install 141.0s
=> [builder 5/6] COPY . . 0.4s
=> [builder 6/6] RUN npm run build 0.4s
=> [stage-1 3/6] COPY --from=builder /app/package.json ./ 0.0s
=> [stage-1 4/6] COPY --from=builder /app/package-lock.json ./ 0.0s
=> [stage-1 5/6] COPY --from=builder /app/dist ./dist 0.1s
=> [stage-1 6/6] COPY --from=builder /app/node_modules ./node_modules 0.2s
=> exporting to image 0.1s
=> => exporting layers 0.1s
=> writing image sha256:8b8a53283b49bad0ef3fd1700035cdd6d13a9a9df7013 0.0s
```

Verify:

docker images

## Step 7: Run Docker Container

docker run -it -p 3000:3000 program2

```
=> exporting to image 0.1s
=> => exporting layers 0.1s
=> writing image sha256:8b8a53283b49bad0ef3fd1700035cdd6d13a9a9df7013 0.0s
=> => naming to docker.io/library/program2 0.0s
1RV24MC109_syeda_ninra@syedaninra-IdeaPad-Gaming-3-15IMH05:~/program2$ sudo docker run -it -p 3000:3000 program2
Server running on port 3000
```

## Step 8: Test the Application

Open browser:

http://localhost:3000

Output:

Hello from multi-stage Docker!

